CC 7.1 - Data Structure Challenge

Thursday, September 11, 2025 3:01 PM

Stack Class

Push(Integer value) - takes the current head of the linked list and replaces it with a newly created node holding the newly added value. This will run at O(1) time.

Pop() - removes the current head of the linked list and returns its value. This runs at O(1) time.

getSize() - returns the value stored in the size variable which runs at O(1) .

isEmpty - checks if the head is null or not which runs at O(1).

Optimization suggestions: I originally implemented the class using an arraylist structure due to its dynamic sizing. However, I noticed that with an arraylist, push and pop operations will run at O(n) time since it would call add(0, value) and remove(0) where 0 is the front of the arraylist. The reason why it would run at O(n) time is because by pushing the value to the front of the arraylist, all of the other values have to be shifted to the right by one. The same could be said about pop where removing the value from the front of the list required that all other values have to be shifted to the left by one. I optimized by using a linked list structure that stored the Node pointer reference of the head which is the top of the stack. That way, push and pop now operate at O(1) time.

Queue Class

Enqueue(Integer value) - Creates a new node holding the new value and appends it to the end of the list using the class's tail variable. Runs at O(1).

Dequeue() - Creates a new node holding the new value and inserts it at the front of the list using the class's head variable. Runs at O(1).

getSize() - returns the value stored in the size variable which runs at O(1) .

isEmpty - checks if the head is null or not which runs at O(1).

Optimization suggestions: I originally also used an arraylist for the queue class. It was not as bad as the stack class in which both its push and pop operations ran in O(n) time because enqueue ran at O(1) time. However, dequeue ran at O(n) time since it also removed the first value in the front of the list, requiring all other values be shifted to the left by one which I thought could be improved. Like the stack class, a linked list data structure was used but the implementation was slightly different. For the queue class, a head variable was created to access the front of the queue and a tail variable was created to access the end of the queue. That way, enqueue (which requires appending to the end of the list) can still run at O(1) time and dequeue (which requires

requires appending to the end of the list) can still run at O(1) time and dequeue (which requires remove from the front of the list) can now run at O(1) time.

Node Class

setNext, getNext, and getValue all run at O(1) time as it involves no looping or lists to loop through.

Main Class

ProcessInput(String input) - uses split method which creates a string array, separates the string by its delimeter, and adds each newly separated string to the array which take O(n) time. Once the string array is created, it also loops through the array (which takes O(n) time), validates the number by trying to parse it into an int (which takes O(n) time), then adds it to the stack or queue (which takes O(1) time). In all, method takes O(n^2) due to looping through the string array and the loop used in validateNumber's parseInt method.

validateNumber(String str) - runs at O(n). After some research I found that Interger.parseInt reads the string character by character to convert it to an integer.

printNumber(int size, Integer number) - runs at O(1) as it involves no looping or lists to loop through.

Main(string[] args) - the while loops for both printing the stack and queue take O(n) but the processInput, as described earlier takes O(n^2). Using the largest time complexity, main runs at O(n^2).

Optimization suggestions: I thought about how main and processInput specifically can be optimized to run at a lower time. One possible optimization is using a while loop that prompts the user for individual numbers until a non number is entered. Instead of the scanner picking up values using nextLine(), it can use the NextInt() method instead and sort the incoming values immediately to its respective stack or queue. I tried to research the time complexity of NextInt in case that it takes the input as a string and then converts it to an integer (in that case would still make main O(n^2). However, I wasn't able to find any solid answers or details of the method's implementation.